
Professional
Report
Bugs
KALI LINUX
"the quieter you become, the more you are able to hear"
Identifying.
Validating.
Securing.
Real Bugs. Real Impact. Real Security.
"GET /api/v1/user
Host: target.com
Authorization:
IDOR | Broken Access Control
SQL Injection
XSS | Cross Site Scripting

Assessment Scope 2
Summary 3
Overview 3
Results 3
Impact 3
Remediation direction 3
Vulnerability writeups 3
Title: 3
Risk rating: 3
Summary: 3
Background 4
Technical Details and Evidence 4
Impact 8
Remediation advises 8

Assessment Scope

To conduct an assessment on the web application with the primary objective of identifying security vulnerabilities present within the application's functionality and demonstrate its security impact. Retrieve a flag as proof.

Summary

Overview

The assessment was conducted within the scope to identify a vulnerability in the web application and obtain the flags associated with the identified weaknesses. Testing focused on the application's user-facing functionality and API requests to identify inconsistencies in how application objects were handled.

Results

The assessment successfully identified and exploited the application's API weakness, resulting in the retrieval of the flag and confirming that the application's object-level access controls could be bypassed.

Impact

The successful exploitation demonstrated that an authenticated user could interact with a hidden application object that was not ordinarily accessible through the user interface. In the context of the CTF, this resulted in unauthorized access to the object containing the challenge flag.

Remediation direction

The application should enforce consistent server-side authorization and object-state validation across API operations to ensure that users can only access or modify objects within their authorized scope.

Vulnerability writeups

Title: Broken Object-Level Authorization (BOLA) / IDOR through an archived post object

Risk rating:

High to Critical

Summary:

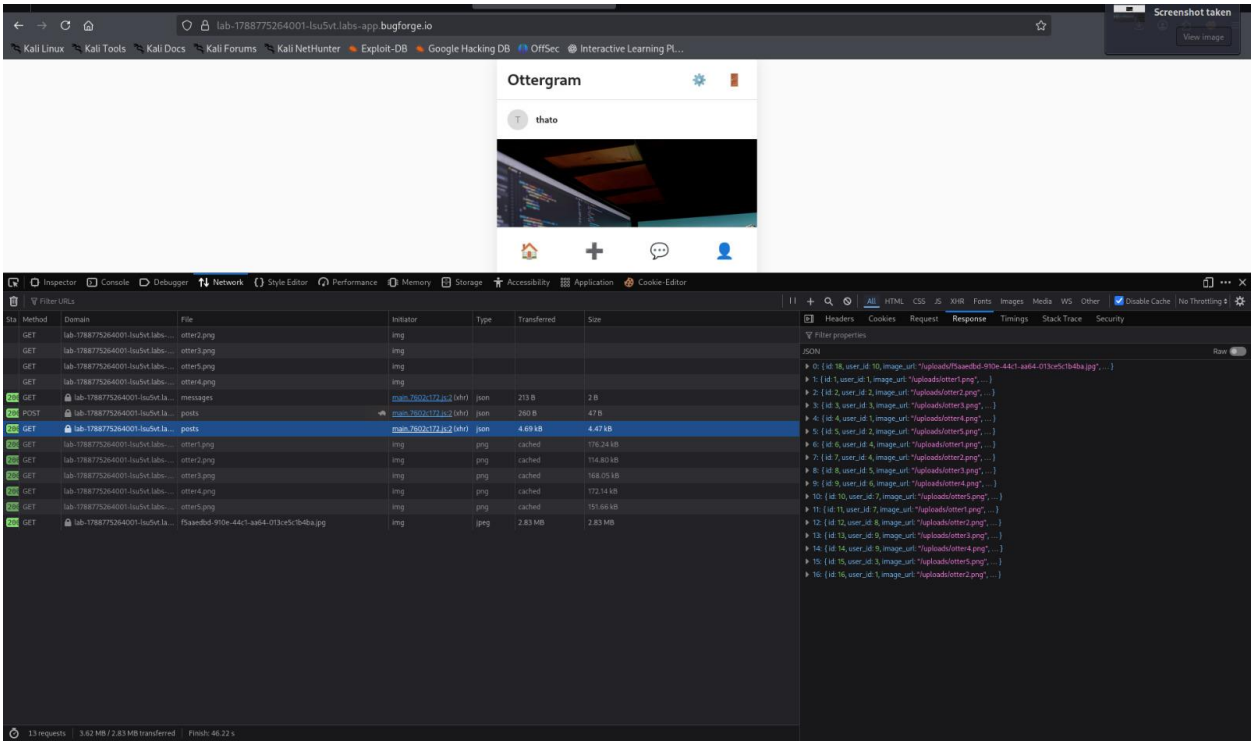
The application was found to contain a Broken Object-Level Authorization (BOLA/IDOR) vulnerability within the posts of API. By manipulating the post identifier in an API request, an authenticated user could interact with a hidden post that was not accessible through the normal user interface, resulting in successful exploitation and retrieval of the flag.

Background

An attacker with a valid user account could potentially manipulate object identifiers within the posts of API to interact with posts outside their intended access scope. Depending on the application's authorization controls and the privileges associated with affected objects, this could allow unauthorized viewing or modification of other users' posts or hidden application data.

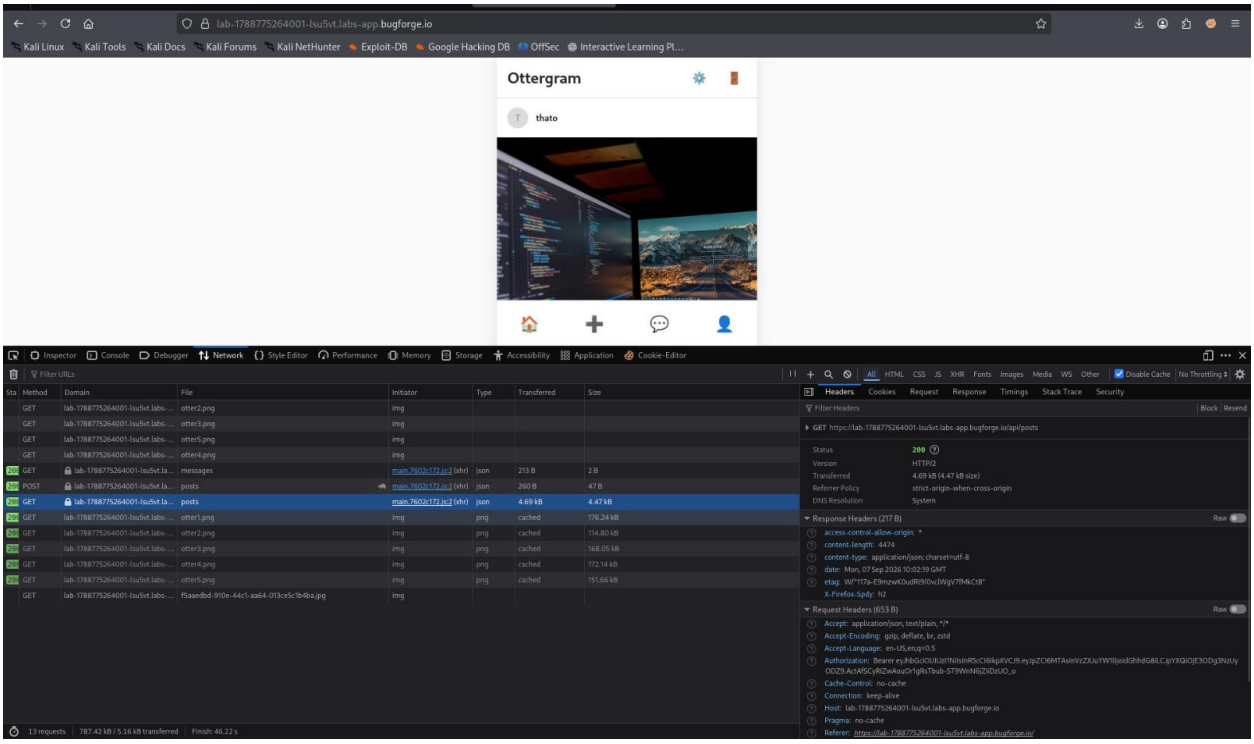
Technical Details and Evidence

During testing, the application's network traffic was monitored to identify API endpoints responsible for post management

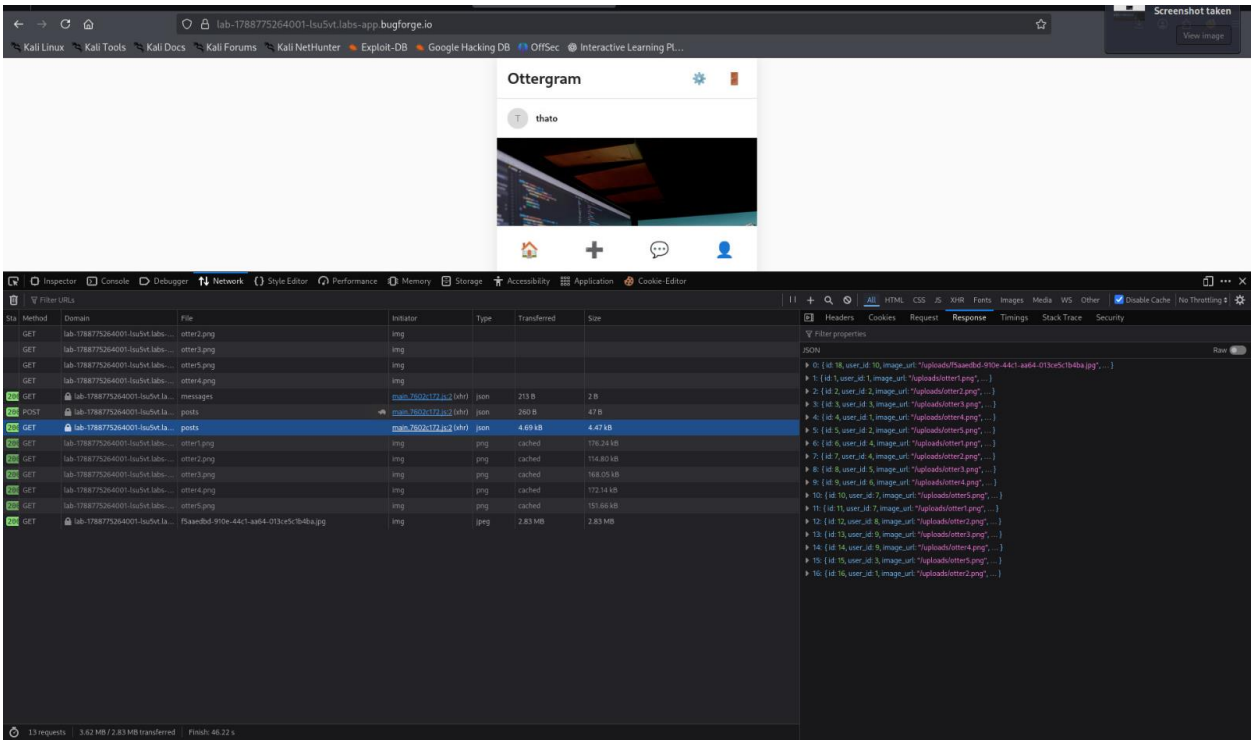


Existing posts were observed through the API, with the highest visible post identifier being 16.

A new post was then created through the application's post functionality. The corresponding API request used the endpoint:

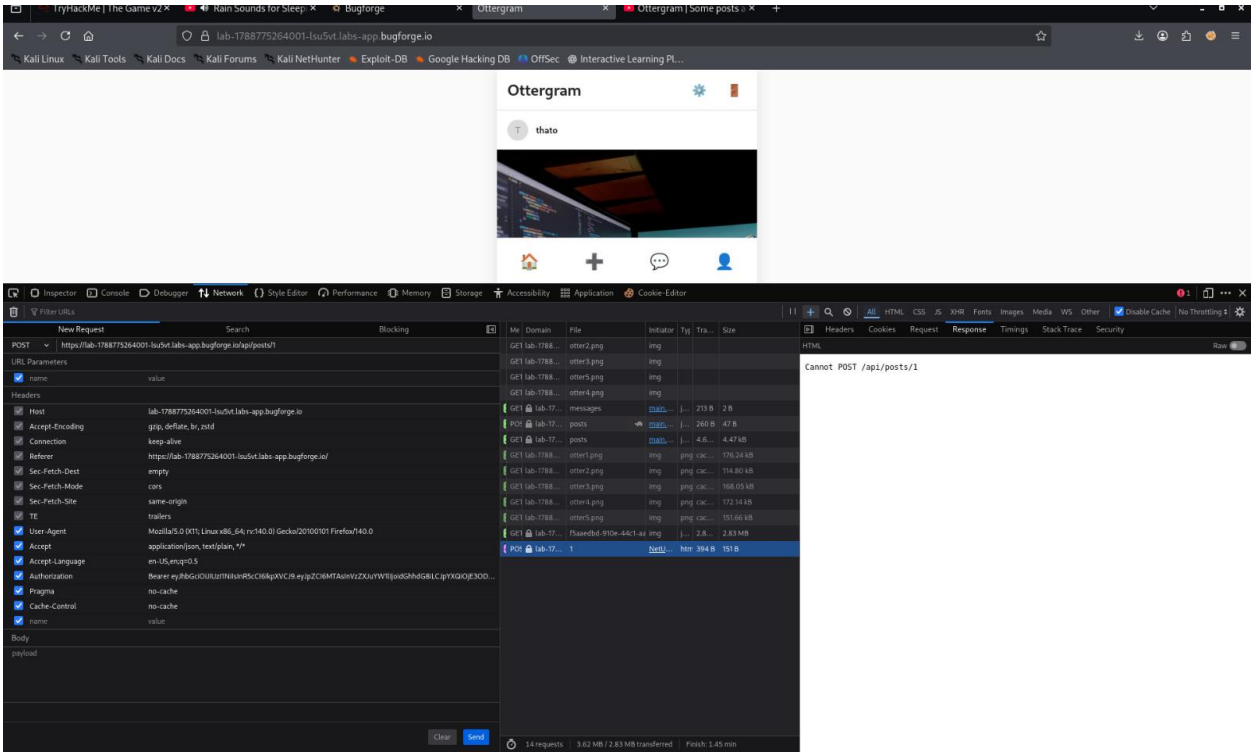


POST /api/posts/18

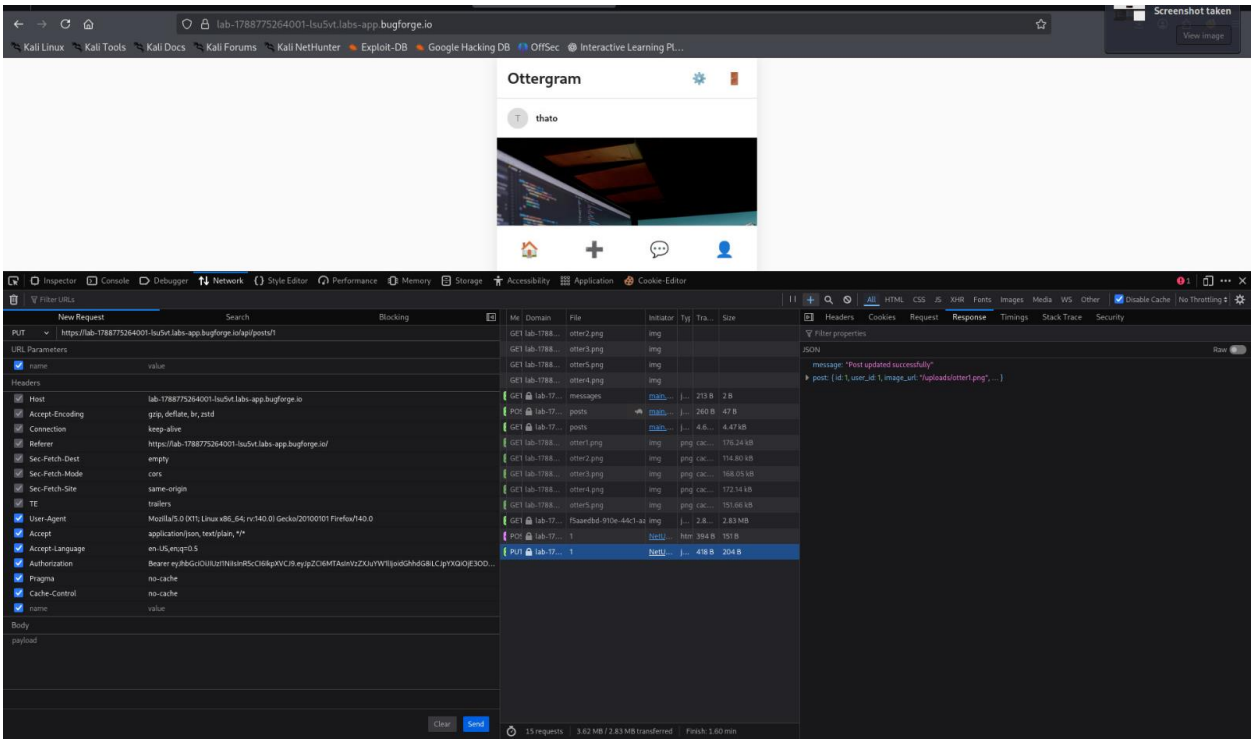


The newly created post was assigned ID 18, indicating that object ID 17 existed but was not exposed through the normal application interface.

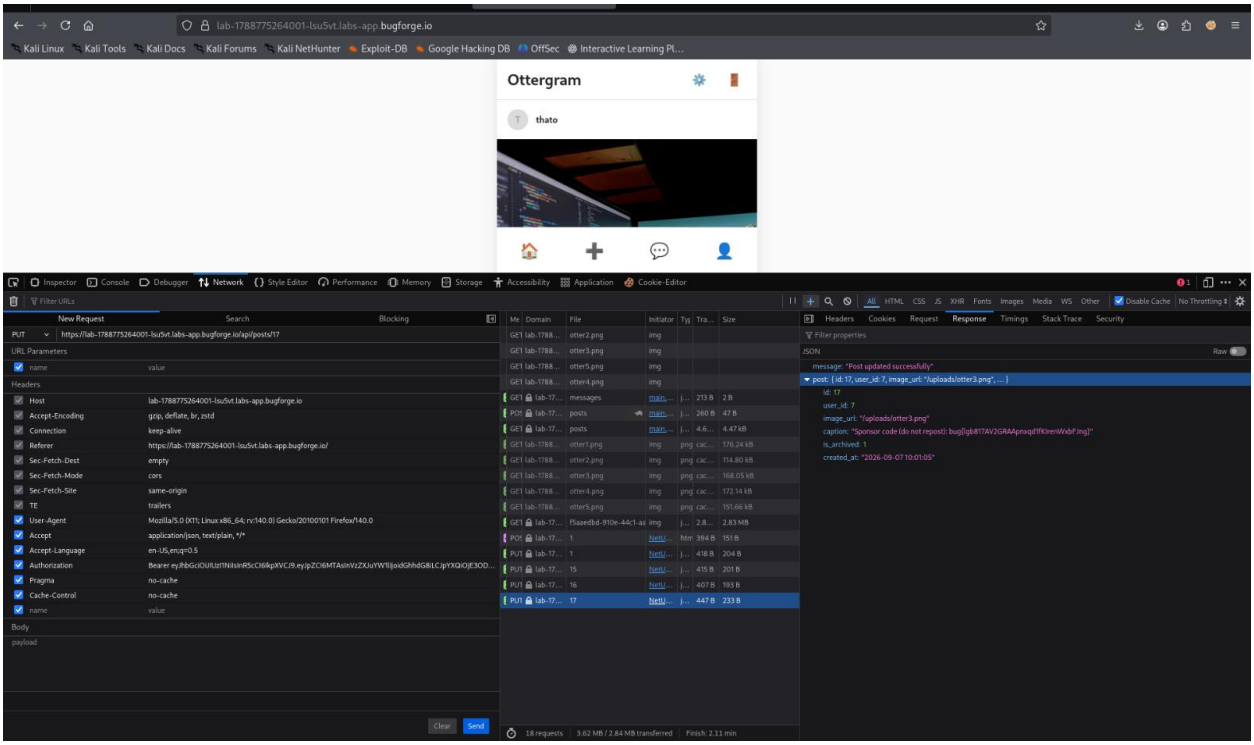
Further testing was performed against the /api/posts/1 endpoint. A GET request to /api/posts/ did not return the hidden object



However, when the request method was changed to PUT and the application responded with a response code of 200.



Manipulating the endpoint posts from 1 to 17 shows.



The server processed the request successfully and returned the flag. This demonstrated that the API permitted interaction with a hidden post through direct manipulation of the object identifier.

Impact

An attacker with an authenticated account could manipulate post identifiers to interact with objects outside their intended access scope. This could expose or modify hidden or unauthorized posts and potentially disclose sensitive application data.

Remediation advises

Implement server-side authorization checks for every post operation. The application should verify that the authenticated user is authorized to access or modify the requested post and that the post's current state permits the requested operation. Object identifiers should not be treated as sufficient authorization.